Sentinel: AI-Powered Real-Time Transaction Screening & Fraud Detection

A Deep-Dive Technical Guide — From Batch File-Upload to Real-Time Money Trail Analysis with AI Agent Verification

***Institution:** IPS Academy, Indore (RGPV University, CSE-AIML Department)*

***Academic Year:** 2026-27*

***Project Type:** B.Tech Major Project-I*

Table of Contents

1. [The Problem We Solved](#1-the-problem-we-solved)
2. [How It Works — Step by Step](#2-how-it-works---step-by-step)
3. [Technology Deep-Dive — Why We Chose Each Tool](#3-technology-deep-dive---why-we-chose-each-tool)
4. [The Confidence Score Algorithm](#4-the-confidence-score-algorithm)
5. [The Verification Agent](#5-the-verification-agent)
6. [Money Trail Detection](#6-money-trail-detection)
7. [Customer Experience](#7-customer-experience)
8. [Production Readiness](#8-production-readiness)
9. [Results & Performance](#9-results--performance)
10. [What's Next](#10-whats-next)
11. [Appendices](#11-appendices)

---

1. The Problem We Solved

The Old Way: Batch File Upload

Before our pivot, the system worked like this:

Customer exports CSV from their bank/CRM

→ Uploads to Sentinel

→ Waits hours or days

→ Gets a PDF report back

→ Manually approves or rejects each transaction

This is slow, error-prone, and completely unsuitable for modern fintech where payments happen in milliseconds.

The New Way: Real-Time Webhook Processing

Our system intercepts every payment as it happens:

Customer connects their CRM (Stripe, Plaid) to Sentinel once

→ Every payment automatically gets screened in real-time

→ Decision comes back in <500ms

→ No manual uploads, no waiting

Why This Matters

Statistic Source

Global AML fines exceeded **$12 billion** in 2024 FinCEN, FFIEC
Synthetic identity fraud surged **311%** (Q1 2024 – Q1 2025) FTC Report
India's UPI handles **15+ billion** monthly transactions RBI
Global AI fraud detection market: **$5.2B → $18.7B by 2030** Grand View Research

Banks and fintechs are bleeding money from fraud while regulators slap them with record fines. The old batch-processing approach can't keep up.

What Makes Sentinel Different

Competitor What They Do What They Miss

Sanctions.io Real-time watchlist screening No money trail analysis, no AI agent
Sardine Merchant fraud detection Closed platform, no open-source
Feedzai Real-time fraud for Tier 1 banks Extremely expensive, not customizable
Checkmarble LLM + rules for fintechs No graph-based ring detection
Sentinel **Screening + Trail + Agent + Decision** **Open-source, explainable, real-time**

Our key innovations:

1. **Agentic AI triage** — autonomous investigation agents, not static rules
2. **Unified platform** — screening, fraud detection, and merchant approval in one system
3. **Open-source core** — competitors are all closed SaaS
4. **Explainability by design** — every decision comes with a full evidence trail
5. **Graph + LLM fusion** — most tools use either graph OR ML; we combine both

---

2. How It Works — Step by Step

The Full Pipeline

■■■
■■■■■■■■■■■

■ CUSTOMER CRM LAYER ■

■ (Stripe · Plaid · your own CRM) — payments originate here ■

■■■
■■■■■■■■■■■

■ Webhook (HTTPS POST)

▼

■■■
■■■■■■■■■■■

■ SENTINEL — REAL-TIME GATEWAY ■

■ ■

■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■

■ ■ Webhook Ingest ■■■■■ Money Trail ■■■■■ Agent Verification ■ ■

■ ■ (Stripe/Plaid) ■ ■ Engine (Neo4j) ■ ■ (Ask · Gather · ■ ■

■ ■ ■ ■ ■ ■ Verify · Decide) ■ ■

■ ■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■
■▼▼▼■
■ Kafka: sentinel.payment Kafka: sentinel.trail Kafka: sentinel.decision■
■■■
■
▼
■■■
■ BANKING / BLOCKCHAIN LAYER ■
■ (Approve / Review / Block decision flows through) ■
■■■

Step 1: Payment Ingestion

When a customer connects their Stripe or Plaid account, every payment that flows through gets intercepted:

What happens when a webhook arrives:

```
POST /api/v1/gateway/payment
Body: {
"type": "payment_intent.succeeded",
"data": {
"object": {
"id": "pi_3N8x...",
"amount": 150000, # in cents = $1,500
"currency": "usd",
"source": { "id": "cus_abc" },
"destination": { "id": "acct_xyz" }
}
}
}
```

→ PaymentWebhookIngestor normalizes this to a PaymentEvent

→ Publishes to Kafka topic: sentinel.payment

The ingestor handles three sources:

• **Stripe**: Parses `payment_intent.succeeded`, `charge.succeeded` events

• **Plaid**: Parses transaction webhooks with `personal_finance_id`

• **Custom**: Any webhook with `payment_id`, `amount`, `currency`, `sender`, `receiver`

Step 2: Money Trail Analysis

Once the payment is in Kafka, the consumer picks it up and traces where the money has been:

Neo4j graph traversal — follow money 5 hops deep

MATCH path = (start {id: $payment_id})-[*1..5]-(connected)

RETURN connected.id, connected.entity_type, length(path)

ORDER BY distance

The **MoneyTrailEngine** detects four patterns:

Pattern What It Looks Like Neo4j Query

Circular Flow Money loops back: A→B→C→A `MATCH path=(start)-[*2..5]-(start) RETURN count(path)`
Structuring Multiple small txns below threshold: 8× $1,200 in 24h `MATCH (s)-[:SENT]->(t) WHERE s.id=$id RETURN count(t), sum(t.amount)`
Layering Money passes through many accounts quickly Graph depth analysis + velocity checks
High-Risk Counterparties Receiver has fraud history `MATCH (a {id:$id})-[]-(b) RETURN b.risk_score`

Step 3: AML Screening

Simultaneously, the existing **ScreeningEngine** checks both parties against sanctions lists:

Screening checks:

■■■ OFAC SDN List (US Treasury)

■■■ UN Consolidated List

■■■ EU Sanctions List

■■■ High-risk jurisdictions (Iran, North Korea, Syria, etc.)

■■■ Fast-name fuzzy matching via Milvus vector embeddings

Example: If the sender is "AHMAD, Mohammed" and the watchlist has "AHMED, Mohamed" — the vector embedding similarity catches this even with the spelling difference.

Step 4: AI Agent Verification

This is the novel part. When the trail and screening produce flags, an AI agent steps in:

Phase 1 — GATHER:

context = {

"payment": {"amount": 15000, "sender": "Alice", ...},

"trail": {"risk_score": 0.6, "circular_flow": True, ...},

"screening": {"flags": ["high_risk_country"], ...}

}

Phase 2 — QUESTION:

The agent generates targeted questions based on gaps:

if trail.risk_score > 0.3:

questions.append("Verify source of funds for sender.")

if screening.flags:

questions.append("Confirm entity identities against watchlists.")

Phase 3 — VERIFY:

Questions are resolved by querying:

• CRM connectors (Stripe/Plaid APIs)

• Knowledge base (historical cases)

• Entity resolution (Milvus fuzzy matching)

Phase 4 — DECIDE:

All signals are aggregated into a final decision.

Step 5: Final Decision

The **DecisionEngine** combines everything:

Final Risk Score = (Trail Risk × 0.35) + (Screening Risk × 0.25) +

(Verification Risk × 0.15) + Policy Flags

Decision Thresholds:

Score ≥ 0.80 → BLOCK (automatic)

Score ≥ 0.50 → REVIEW (human checks evidence)

Score < 0.50 → APPROVE (money flows through)

Sanctions match → BLOCK (hard override, no exceptions)

The decision is published to Kafka topic `sentinel.decision` and the banking/blockchain layer receives the result.

---

3. Technology Deep-Dive — Why We Chose Each Tool

3.1 FastAPI (Not Flask or Django)

Factor	FastAPI	Flask	Django

Async support	Native	Plugin	Partial
Auto OpenAPI docs	Yes (`/docs`)	No	No
Request throughput	200K+ req/s	~50K req/s	~30K req/s
Type safety	Pydantic validation	Manual	Forms
Learning curve	Moderate	Easy	Steep

Why FastAPI: Our system processes payments in real-time. Every millisecond matters. FastAPI's async-native architecture lets us handle 200K+ requests per second, and the auto-generated `/docs` means our API is self-documenting — critical for customer integration.

3.2 Apache Kafka (Not REST Polling or Redis Streams)

Factor	Kafka	Redis Streams	REST Polling

Throughput	1M+ msgs/sec	~100K msgs/sec	Limited by HTTP
Replayability	Yes (offsets)	Limited	No
Durability	Disk-backed	Memory-first	N/A
Decoupling	Full producer/consumer	Partial	None
Backpressure	Built-in	Manual	N/A

Why Kafka: When a payment hits, we need to process it through 4 stages (trail → screen → verify → decide) without blocking the webhook response. Kafka decouples ingestion from processing — the webhook returns "received" immediately, and the consumer processes at its own pace. If Kafka goes down, we fall back to synchronous processing (graceful degradation).

The topics we use:

• `sentinel.payment` — incoming webhooks

• `sentinel.trail` — trail analysis results

• `sentinel.decision` — final APPROVE/REVIEW/BLOCK

3.3 PostgreSQL (Not MongoDB)

Factor	PostgreSQL	MongoDB

| ---------- |
| ACID compliance Full Eventual |
| JSONB support Yes (indexed) Native BSON |
| Complex queries SQL (mature) MongoDB Query |
| Extensibility Extensions (pgvector, PostGIS) Limited |
| Migrations Alembic (mature) Limited |

Why PostgreSQL: Financial data demands ACID compliance — every transaction must be atomic. PostgreSQL also gives us JSONB for flexible schema (merchant profiles vary) and the `pgvector` extension for in-database vector search. Our data is relationship-heavy (entities → transactions → trails), and SQL handles joins better than MongoDB's document model.

3.4 Neo4j (Not PostgreSQL for Graph)

Factor Neo4j PostgreSQL + pg_graphql

Graph traversal Native Cypher SQL JOINs (slow for deep)
Fraud ring detection Optimized Query gets complex fast
Circular flow `MATCH path=(a)-[*]-(a)` Recursive CTE (limited depth)
Performance O(1) neighbor lookup O(n²) join explosion

Why Neo4j: Money trails ARE graphs. A person (node) sends money to another person (node) via a transaction (edge). Finding circular flows (A→B→C→A) is one Cypher query. In PostgreSQL, it's a recursive CTE that chokes after 3-4 hops. Our trail engine traverses up to 5 hops deep — Neo4j handles this natively.

The key Cypher queries:

-- Circular flow detection

MATCH path = (start {id: $payment_id})-[*2..5]-(start)

RETURN count(path)

-- Shared device fingerprints (fraud ring indicator)

MATCH (a), (b)

WHERE a.id IN $entity_ids AND b.id IN $entity_ids

AND a.device_fingerprint = b.device_fingerprint

RETURN a.id, b.id

3.5 Redis (Not Python Dict)

Factor Redis Python Dict

Persistence Yes (AOF/RDB) No (in-memory only)
TTL support Native Manual
Distributed Yes (cluster mode) Process-local
Data structures Lists, Sets, Sorted Sets Dicts, Lists
Latency ~1ms ~0.001ms

Why Redis: We use Redis for two things:

1. **Trail result caching** — Neo4j queries are slow (50-200ms). We cache trail analysis for 5 minutes so repeated queries are instant.

2. **Rate limiting** — We need to track how many requests each API key makes per minute. Redis' atomic INCR + EXPIRE is perfect for this.

A Python dict would work for a demo, but it dies when the process restarts and can't be shared across multiple API instances.

3.6 Milvus (Not ChromaDB)

Factor Milvus ChromaDB

Scale Billion+ vectors ~10M vectors
Hybrid search Vector + metadata + full-text Vector only
Deployment Standalone or distributed Embedded or server
Production readiness Enterprise-grade Emerging

Why Milvus: Sanctions screening needs fuzzy name matching — "AHMAD, Mohammed" should match "M. Ahmed" even with spelling differences. Milvus stores entity embeddings and supports hybrid search (vector similarity + metadata filter for country/list source). ChromaDB is great for development but can't handle the scale we need.

3.7 Prometheus + Grafana (Not CloudWatch or Datadog)

Factor Prometheus + Grafana CloudWatch Datadog

Cost Free (self-hosted) Pay per metric $15+/host/month
Query language PromQL (powerful) CloudWatch Logs DQL
Integration OpenTelemetry native AWS-only Multi-cloud
Alerts Alertmanager CloudWatch Alarms Synthetic monitors

Why Prometheus: We track 12 key metrics:

• `sentinel_request_duration_seconds` — p50/p95/p99 latency

• `sentinel_decisions_total` — APPROVE/REVIEW/BLOCK counts

• `sentinel_fraud_detection_rate` — true positive rate

• `sentinel_kafka_lag_seconds` — consumer health

Grafana visualizes these in real-time dashboards. CloudWatch would cost us $200+/month; Prometheus is free.

3.8 OmniRoute (Not Direct API Calls)

Factor OmniRoute Direct API

Cost Free tier (90+ models) $0.002-0.03 per 1K tokens
Fallback Automatic model rotation Manual retry logic
Models available 290 providers Single provider
Abstraction OpenAI-compatible API Provider-specific SDKs

Why OmniRoute: The verification agent needs an LLM to generate questions and evaluate evidence. OmniRoute gives us access to 290+ models through a single OpenAI-compatible API. If the primary model rate-limits, it auto-falls back to another. Direct API calls would require us to manage 5+ provider keys and retry logic ourselves.

---

4. The Confidence Score Algorithm

The Formula

The DecisionEngine combines four signal sources into a single risk score (0.0 to 1.0):

Risk Score = (Trail_Risk × 0.35) + (Screening_Risk × 0.25) +

(Verification_Risk × 0.15) + Policy_Flags

Where:

Verification_Risk = 1.0 - Verification_Confidence

Policy_Flags = sum of weighted flag penalties

Policy Flag Weights

Flag Weight Why

Circular flow detected +0.30 Classic money laundering pattern
Structuring detected +0.20 Avoiding reporting thresholds
Sanctions match +0.40 Legal requirement to block
Each high-risk counterparty +0.15 (max +0.30) Known fraud associations
Layering detected +0.25 Complex obfuscation pattern
Shell company indicators +0.35 Fake entity red flag
Each screening flag +0.10 (max +0.20) AML rule violations

Decision Thresholds

Score ≥ 0.80 → BLOCK (automatic, no human review)

Score ≥ 0.50 → REVIEW (human compliance officer decides)

Score < 0.50 → APPROVE (money flows through)

SANCTIONS MATCH → BLOCK (hard override, regardless of score)

Worked Example

Scenario: A $15,000 payment from "Alice (Iran)" to "Bob (Syria)"

Signal Source Raw Value Weighted Contribution

Trail risk (Neo4j) 0.7 (circular flow + 2 high-risk counterparties) 0.7 × 0.35 = **0.245**
Screening risk (OFAC) 0.6 (high-risk jurisdictions) 0.6 × 0.25 = **0.150**
Verification confidence 0.4 (agent couldn't resolve questions) (1.0 - 0.4) × 0.15 = **0.090**
Policy flags circular_flow(+0.30) + high_risk_country(+0.15×2) **+0.600**
TOTAL **1.085 → clamped to 1.0**

Decision: BLOCK (score 1.0, confidence 0.95)

Evidence: "Circular money flow detected. Sender from Iran, receiver from Syria. 2 high-risk counterparties identified. Sanctions match: Entity Bob on OFAC list."

Why This Formula Works

1. **Trail risk dominates (35%)** — Money trail analysis is the most predictive signal. If money loops back or passes through suspicious entities, it's almost always fraud.

2. **Screening is secondary (25%)** — Watchlist matches are important but have false positives (common names).

3. **Verification fills gaps (15%)** — When the agent can't resolve questions (low confidence), that uncertainty adds risk.

4. **Flags are penalties (25%)** — Detected patterns (circular flow, structuring) are strong indicators that push scores up aggressively.

---

5. The Verification Agent

The 4-Phase Pipeline

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Verification Pipeline ■
■ ■
■ Phase 1: GATHER Phase 2: QUESTION ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■
■ ■ Collect all ■ ■ Generate ■ ■
■ ■ context ■■■■■■■■■■■■ targeted ■ ■
■ ■ from trail, ■ ■ questions ■ ■
■ ■ screen, ■ ■ based on ■ ■
■ ■ fraud data ■ ■ gaps ■ ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■
■ ■ ■ ■
■ ▼ ▼ ■
■ Phase 3: VERIFY Phase 4: DECIDE ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■
■ ■ Answer ■ ■ Aggregate ■ ■
■ ■ questions ■■■■■■■■■■■■ all signals ■ ■
■ ■ via CRM, ■ ■ into final ■ ■
■ ■ DB, KB ■ ■ decision ■ ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■
■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Phase 1: Gather

The agent collects everything it can about the payment:

```
context = {
"payment": {
"amount": 15000,
"sender": {"entity_id": "s1", "name": "Alice", "country": "IR"},
"receiver": {"entity_id": "r1", "name": "Bob", "country": "SY"},
},
"trail": {
"aggregated_risk_score": 0.7,
"nodes": [ ... ], # 12 connected entities
"flags": {
"circular_flow": True,
"high_risk_counterparties": ["entity_3", "entity_7"],
}
},
"screening": {
"risk_score": 0.6,
```

"flags": ["high_risk_country"],

"matched_entities": [{"name": "Bob", "list_source": "OFAC"}],

}

}

Phase 2: Question

The agent generates questions based on what's MISSING, not what's present:

If trail risk is high but we don't know the source of funds:

if trail.risk_score > 0.3:

questions.append({

"text": "Verify source of funds for sender. Amount is $15,000 from Iran.",

"type": "fact_check",

"priority": "HIGH"

})

If screening found a match but we need confirmation:

if screening.matched_entities:

questions.append({

"text": f"Confirm identity of {names}. Matched on {lists} watchlist.",

"type": "context_gather",

"priority": "HIGH"

})

Phase 3: Verify

Each question is answered by querying the right source:

Question Type Answer Source Example

Fact check CRM connector (Stripe/Plaid) "What is the account age for entity X?"
Context gather Knowledge base (PostgreSQL) "Has this entity been flagged before?"
Entity resolution Milvus vector search "Is 'M. Ahmed' the same as 'Mohammed Ahmed'?"
Document request RAG pipeline "Extract business registration from uploaded doc"

Phase 4: Decide

Three strategies handle different confidence levels:

Strategy When to Use How It Works

RuleBased Clear cases (score < 0.1 or > 0.9) Threshold rules only, fast
Agentic Complex cases (0.1 < score < 0.9) LLM evaluates full context
Hybrid Default Rules first, agent for edge cases

Hybrid strategy in action:

if risk_score >= 0.9 or risk_score <= 0.1:

return rule_based.decide(score, flags, context) # Fast, deterministic

else:

return agentic.decide(score, flags, context) # LLM reasoning

---

6. Money Trail Detection

What Is a Money Trail?

A money trail is the path money takes through the financial system. In our system, every transaction is a node in a graph, and relationships between entities (shared devices, same addresses, frequent transfers) are edges.

```
■■■■■■■■■■■■ ■ Alice ■ ← sender (entity_id: a1) ■■■■■■■■■■■■ ■ SENT $5,000 ■■■■■▼■■■■■ ■ Bob ■ ← intermediary
(entity_id: b1) ■■■■■■■■■■■■ ■ SENT $4,800 ■■■■■▼■■■■■ ■ Charlie ■ ← receiver (entity_id: c1) ■■■■■■■■■■■■ ■
SENT $4,900 (looping back!) ■■■■■▼■■■■■ ■ Alice ■ ← SAME AS SENDER! ■■■■■■■■■■■■
```

This is a **circular flow** — money started with Alice and ended back with Alice, passing through Bob and Charlie. This is a classic money laundering pattern.

Detection Algorithms

-- Find if money loops back to the sender within 5 hops

MATCH path = (start {id: $payment_id})-[*2..5]-(start)

WHERE length(path) > 1

RETURN count(path) AS cycle_count

If `cycle_count > 0`, circular flow is detected. This adds +0.30 to the risk score.

Structuring (aka "smurfing") is when someone splits a large transaction into multiple small ones to stay below reporting thresholds (e.g., $10,000 in the US).

-- Find if same sender has many small transactions

MATCH (sender)-[:SENT]->(t)

WHERE sender.id = $payment_id

RETURN count(t) AS txn_count, sum(t.amount) AS total_amount

If `txn_count > 5` AND `total_amount > 10000` within 24 hours → structuring detected (+0.20 to risk).

-- Check if receiver has been linked to fraud before

MATCH (receiver {id: $entity_id})-[]-(connected)

RETURN connected.id, connected.risk_score

Entities with `risk_score > 0.7` are flagged as high-risk counterparties (+0.15 each, max +0.30).

Why Graph DB Beats SQL for This

In PostgreSQL, finding a 5-hop circular flow requires a recursive CTE:

WITH RECURSIVE trail AS (

SELECT sender_id, receiver_id, 1 AS depth

FROM transactions WHERE sender_id = 'alice'

UNION ALL

SELECT t.sender_id, t.receiver_id, depth + 1

FROM transactions t JOIN trail ON t.sender_id = trail.receiver_id

WHERE depth < 5

)

SELECT * FROM trail WHERE receiver_id = 'alice';

This works for 2-3 hops but becomes extremely slow at 5 hops. Neo4j's native graph traversal is O(1) per hop — it just follows pointers, no joins needed.

---

7. Customer Experience

What the Customer Sees

A fintech company (e.g., a payment processor) connects Sentinel to their Stripe account. Here's their daily experience:

■■■

■ Sentinel Dashboard ■

■■■

■ ■ Today's Activity ■ ■ Recent Alerts ■

■ Approved: 10,200 ■ 1. $15,000 → BLOCK (circular flow) ■

■ Reviewed: 1,980 ■ 2. $8,500 → REVIEW (sanctions hit) ■

■ Blocked: 270 ■ 3. $3,200 → REVIEW (structuring) ■

■ ■ ■

■ ■ Avg Processing: 340ms■ ■ This Month ■

■ ■ Fraud caught: 47 transactions ■

■ ■ Detection Rate: 94% ■ Sanctions matches: 12 entities ■

■■■

Alert Workflow

When a payment is flagged:

1. **System** holds the payment and generates an evidence dossier

2. **Compliance officer** gets a notification (email + dashboard alert)

3. **Officer reviews** the evidence:

• Trail visualization (graph of connected entities)

• Screening results (which watchlists matched)

• Agent's questions and answers

• Confidence score and reasoning

4. **Officer decides**: Release or Block

5. **Decision is logged** with full audit trail (required by regulation)

Configuration Options

Customers can adjust their risk tolerance:

Be more strict

policy = DecisionPolicy(block_threshold=0.7, review_threshold=0.4)

Be more lenient (fewer false positives)

policy = DecisionPolicy(block_threshold=0.9, review_threshold=0.6)

---

8. Production Readiness

Infrastructure (docker-compose.yml)

Our system runs 10 services:

Service Purpose Port
----
api FastAPI application 8000
postgres Primary database 5432
redis Cache + rate limiting 6379
kafka Event streaming 9092
zookeeper Kafka coordination 2181
neo4j Graph database 7474/7687
milvus Vector database 19530
etcd Milvus metadata 2379
prometheus Metrics collection 9090
grafana Dashboards 3000

One-Command Startup

git clone https://github.com/celestialgeeks/sentinel-fintech.git

cd sentinel-fintech

docker compose up -d

Wait 60 seconds for services to start

curl http://localhost:8000/health

→ {"status": "ok", "service": "sentinel-api"}

Scaling to Production

For production, we deploy on Kubernetes:

• **HPA** (Horizontal Pod Autoscaler) scales API replicas based on CPU

• **Persistent Volumes** for PostgreSQL and Milvus data

• **ConfigMaps** for environment-specific settings

• **Ingress** for HTTPS termination

Monitoring

Prometheus scrapes metrics every 15 seconds. Grafana dashboards show:

• Request latency (p50, p95, p99)

• Throughput (payments/sec)

• Kafka consumer lag

• Error rates by endpoint

• Fraud detection rate over time

Alerts fire when:

• p99 latency > 1 second

• Error rate > 1%

• Kafka lag > 5 minutes

• Fraud detection rate drops below 85%

---

9. Results & Performance

Current Status

Metric Value

API Endpoints 18 (7 new + 5 existing + 6 health/status)
Tests 25 passing (gateway, decision, verification)
Coverage Target 40% (current: 18% — needs more tests)
Processing Target < 500ms per payment
Model Accuracy Target > 90% true positive rate
False Positive Target < 15%

API Endpoints

POST /api/v1/gateway/payment — Ingest payment webhook

POST /api/v1/gateway/trail/{id} — Analyze money trail

GET /api/v1/gateway/trail/{id}/status — Trail analysis status

GET /api/v1/gateway/health — Health check

POST /api/v1/verification/run — Run full verification

POST /api/v1/verification/question/{id} — Ask specific question

GET /api/v1/verification/status/{id} — Verification status

GET /api/v1/verification/decision/{id} — Final decision

POST /api/v1/decision/run — Run decision engine

GET /api/v1/decision/health — Decision health check

Existing endpoints (unchanged)

POST /api/v1/screening/transactions

GET /api/v1/screening/transactions/{id}

POST /api/v1/fraud/check

GET /api/v1/fraud/alerts

POST /api/v1/fraud/alerts/{id}/triage

POST /api/v1/merchant/onboard

GET /api/v1/merchant/status/{id}

GET /health

---

10. What's Next

Phase 6: Real Data Population

• Seed Neo4j with 100K+ synthetic transactions for realistic testing

• Populate Milvus with real sanctions list embeddings (500K+ entities)

• Connect to live Stripe test mode for end-to-end validation

Phase 7: Live CRM Integration

• Replace mock Stripe/Plaid connectors with real API calls

• Add webhook signature verification (security)

• Implement idempotency keys (prevent duplicate processing)

Phase 8: ML Model Training

• Train XGBoost fraud model on synthetic + public datasets

• Fine-tune LLM for better question generation

• Implement model monitoring (drift detection)

Phase 9: Mobile App

• iOS/Android app for compliance officers

• Push notifications for high-severity alerts

• Offline case review capability

Phase 10: Multi-Jurisdiction

• Add RBI (India) specific rules

• EU AMLD 6th directive compliance

• FATF travel rule support for crypto

---

11. Appendices

Appendix A: Complete API Reference

Method Path Request Body Response

POST `/api/v1/gateway/payment` `{type, data: {object: {...}}}` `{success, data: {payment_id, source}}`
POST `/api/v1/gateway/trail/{id}` — `{success, data: TrailAnalysis}`
GET `/api/v1/gateway/trail/{id}/status` — `{payment_id, status, has_decision}`
POST `/api/v1/verification/run` `{payment_event, trail_analysis, screening_result}` `VerificationDecision`
POST `/api/v1/decision/run` `{payment_id, trail_analysis, screening_result, verification_result}` `GatewayResponse`

Appendix B: Database Schema (PostgreSQL)

-- Core entities

CREATE TABLE entities (

id VARCHAR PRIMARY KEY,

name VARCHAR NOT NULL,

entity_type VARCHAR, -- 'person', 'merchant', 'blockchain_wallet'

country VARCHAR,

risk_score FLOAT DEFAULT 0.0,

attributes JSONB,

created_at TIMESTAMP DEFAULT NOW()

);

CREATE TABLE transactions (

id VARCHAR PRIMARY KEY,

sender_id VARCHAR REFERENCES entities(id),

receiver_id VARCHAR REFERENCES entities(id),

```
amount DECIMAL(15,2) NOT NULL,
currency VARCHAR(3),
timestamp TIMESTAMP NOT NULL,
source VARCHAR, -- 'stripe', 'plaid', 'custom'
created_at TIMESTAMP DEFAULT NOW()
);
CREATE TABLE screening_results (
id VARCHAR PRIMARY KEY,
transaction_id VARCHAR REFERENCES transactions(id),
risk_score FLOAT,
flags JSONB,
matched_entities JSONB,
recommendation VARCHAR,
created_at TIMESTAMP DEFAULT NOW()
);
CREATE TABLE decisions (
id VARCHAR PRIMARY KEY,
payment_id VARCHAR REFERENCES transactions(id),
decision VARCHAR NOT NULL, -- 'APPROVE', 'REVIEW', 'BLOCK'
confidence FLOAT,
risk_score FLOAT,
reasoning TEXT,
evidence JSONB,
created_at TIMESTAMP DEFAULT NOW()
);
```

Appendix C: Kafka Topic Schema

```
Topic: sentinel.payment
Key: payment_id (string)
Value: {
"payment_id": "pi_3N8x...",
"timestamp": "2026-08-21T10:30:00Z",
"amount": 15000.00,
"currency": "USD",
"sender": {"entity_id": "s1", "name": "Alice", "country": "US"},
"receiver": {"entity_id": "r1", "name": "Bob", "country": "UK"},
"source": "stripe",
"metadata": {"event_type": "payment_intent.succeeded"}
}
Topic: sentinel.trail
Key: payment_id
Value: {
"payment_id": "pi_3N8x...",
"trail_depth": 3,
```

"nodes": [...],
"flags": {"circular_flow": true, "structuring": false},
"aggregated_risk_score": 0.75,
"summary": "Circular flow detected. 12 nodes found."
}
Topic: sentinel.decision
Key: payment_id
Value: {
"payment_id": "pi_3N8x...",
"decision": "BLOCK",
"confidence": 0.95,
"risk_score": 0.85,
"reasoning": "Circular flow + sanctions match",
"evidence": ["Circular money flow detected", "Sanctions match: Entity Bob"],
"processed_at": "2026-08-21T10:30:01Z"
}

Appendix D: Neo4j Cypher Queries

```
-- 1. Follow money trail (5 hops)
MATCH path = (start {id: $payment_id})-[*1..5]-(connected)
RETURN connected.id AS entity_id,
connected.entity_type AS type,
labels(connected) AS labels,
length(path) AS distance,
connected.risk_score AS risk_score,
connected.name AS name,
connected.attributes AS attributes
ORDER BY distance
-- 2. Circular flow detection
MATCH path = (start {id: $payment_id})-[*2..5]-(start)
WHERE length(path) > 1
RETURN count(path) AS cycle_count
-- 3. Structuring detection
MATCH (sender)-[:SENT]->(t)
WHERE sender.id = $payment_id
RETURN count(t) AS txn_count, sum(t.amount) AS total_amount
-- 4. Fraud ring detection
MATCH (start {id: $entity_id})-[*1..2]-(connected)
RETURN connected.id, connected.entity_type, labels(connected)
-- 5. Shared device fingerprints
MATCH (a), (b)
WHERE a.id IN $entity_ids AND b.id IN $entity_ids
AND a.id < b.id
AND a.device_fingerprint IS NOT NULL
```

AND a.device_fingerprint = b.device_fingerprint

RETURN a.id AS entity_a, b.id AS entity_b, a.device_fingerprint AS shared_value

Appendix E: Configuration (.env.example)

Database

DATABASE_URL=postgresql+asyncpg://sentinel:sentinel_pass@localhost:5432/sentinel

Redis

REDIS_URL=redis://:sentinel_redis_pass@localhost:6379/0

Kafka

KAFKA_BOOTSTRAP_SERVERS=localhost:9092

KAFKA_SCHEMA_REGISTRY=http://localhost:8081

Neo4j

NEO4J_URI=bolt://localhost:7687

NEO4J_USER=neo4j

NEO4J_PASSWORD=sentinel_neo4j_pass

Milvus

MILVUS_URI=http://localhost:19530

OmniRoute (LLM)

OMNIROUTE_BASE_URL=http://127.0.0.1:20128/v1

OMNIROUTE_API_KEY=sk-9fe...6522

JWT (for API auth)

JWT_SECRET=change-this-to-a-secure-random-string

JWT_ALGORITHM=HS256

Environment

ENVIRONMENT=development

Document generated for B.Tech Major Project-I, CSE-AIML, IPS Academy, Indore (RGPV University), Academic Year 2026-27.